Stack Definitivo (Optimizado para Sistema Interno)

Frontend

```
React + TypeScript + Vite
├── React Router DOM (rutas)
├── Tailwind CSS (estilos)
├── Zustand o Context API (estado global)
├── Socket.io-client (tiempo real)
└── Axios (HTTP requests)
```

Backend

```
NestJS + TypeScript
├── Prisma ORM
├── Socket.io (WebSocket server)
├── Passport + JWT (autenticación)
├── bcrypt (hash contraseñas)
└── Winston (logging)
```

Base de Datos

```
PostgreSQL 15+
└── Redis (sesiones + caché)
```

```
┌──────────────────────────────────────────────┐
│   FRONTEND (React + Vite + TS)               │
│                                              │
│  • SPA (Single Page Application)             │
│  • React Router para navegación              │
│  • Auth Context para sesión                  │
│  • Socket.io para notificaciones real        │
│                                              │
└──────────────────────┬───────────────────────┘
                       │
                       │ HTTP REST + WebSocket
                       │
```

```
┌──────────────────────▼──────────────────────────────────────┐
│     BACKEND (NestJS + TypeScript)      │
│                                        │
│  • API REST endpoints                  │
│  • WebSocket Gateway (Socket.io)       │
│  • JWT Authentication                  │
│  • RBAC (Roles + Permissions)          │
│  • Audit Logs                          │
│                                        │
└────────────────────┬────────────────────────────────────────┘
                     │
                     │ Prisma ORM
                     │
┌──────────────────────▼──────────────────────────────────────┐
│    PostgreSQL + Redis                  │
│                                        │
│  PostgreSQL:                           │
│  • Users, Roles, Permissions           │
│  • Rentals, Credits, Cash              │
│  • Audit Logs                          │
│                                        │
│  Redis:                                │
│  • User sessions (JWT blacklist)       │
│  • Real-time notifications queue       │
│  • Cache de consultas frecuentes       │
│                                        │
└────────────────────────────────────────────────────────────┘
```

Estructura de Proyecto Frontend

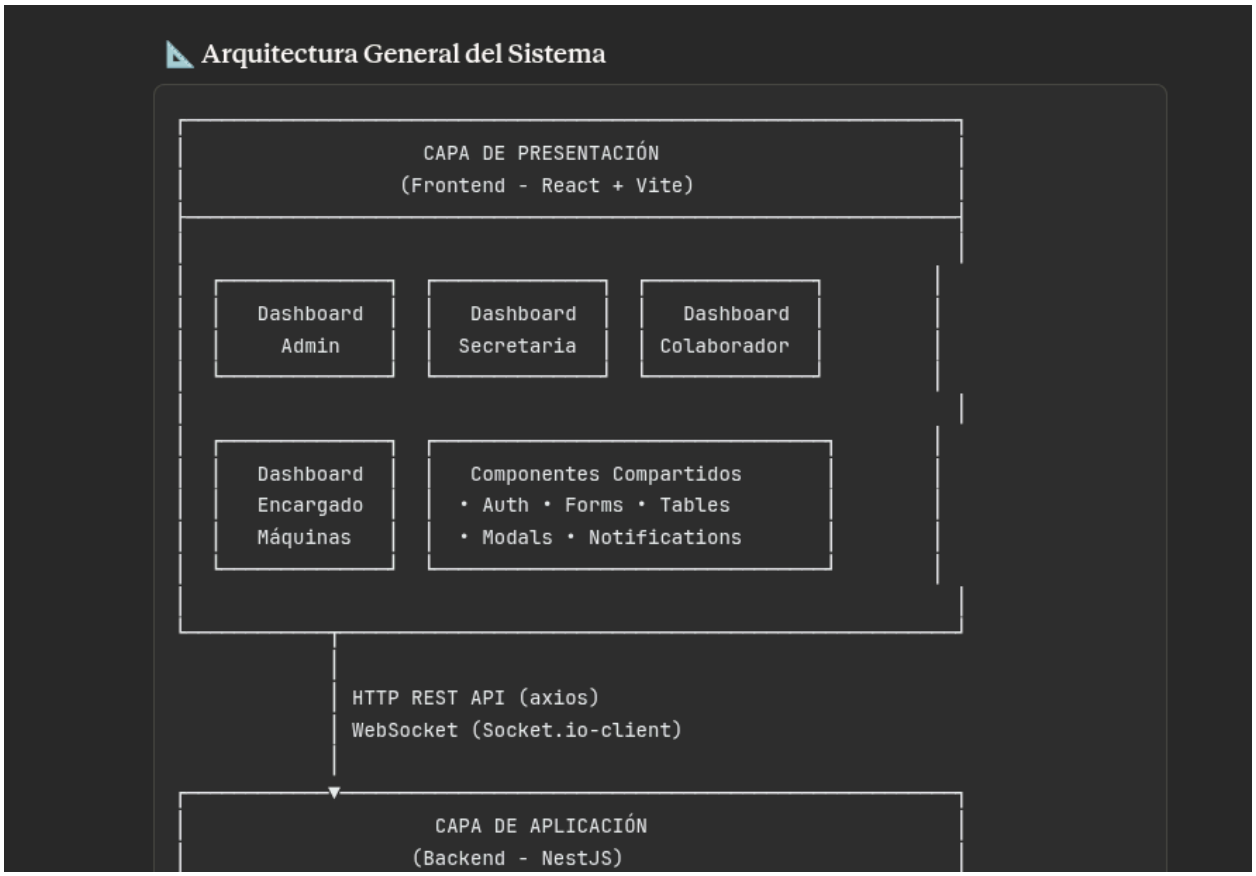
```
frontend/
├── src/
│   ├── main.tsx
│   ├── App.tsx
│   ├── routes/
│   │   ├── AppRouter.tsx
│   │   └── ProtectedRoute.tsx
│   ├── pages/
│   │   ├── Login.tsx
│   │   ├── admin/
│   │   │   ├── Dashboard.tsx
│   │   │   ├── UserManagement.tsx
```

```
│   │   │   └── RoleManagement.tsx
│   │   ├── secretaria/
│   │   │   ├── Dashboard.tsx
│   │   │   ├── RentalApprovals.tsx
│   │   │   └── CreditApprovals.tsx
│   │   ├── colaborador/
│   │   │   ├── Dashboard.tsx
│   │   │   ├── RequestCredit.tsx
│   │   │   └── RequestCash.tsx
│   │   └── encargado-maquinas/
│   │       ├── Dashboard.tsx
│   │       └── RentalRequests.tsx
│   ├── components/
│   │   ├── layouts/
│   │   │   ├── DashboardLayout.tsx
│   │   │   └── Sidebar.tsx
│   │   ├── common/
│   │   │   ├── Button.tsx
│   │   │   ├── Modal.tsx
│   │   │   └── Table.tsx
│   │   └── notifications/
│   │       └── NotificationToast.tsx
│   ├── hooks/
│   │   ├── useAuth.ts
│   │   ├── useSocket.ts
│   │   └── usePermissions.ts
│   ├── services/
│   │   ├── api.ts
│   │   ├── auth.service.ts
│   │   ├── socket.service.ts
│   │   └── rentals.service.ts
│   ├── store/
│   │   ├── authStore.ts
│   │   └── notificationStore.ts
│   ├── types/
│   │   ├── user.types.ts
│   │   ├── rental.types.ts
│   │   └── api.types.ts
│   └── utils/
│       ├── formatters.ts
│       └── validators.ts
├── index.html
├── vite.config.ts
├── tsconfig.json
```

```
├── tailwind.config.js
└── package.json
```

Arquitectura Completa del Sistema de Ferretería 🏗️

📐 Arquitectura General del Sistema



```
CAPA DE APLICACIÓN
(Backend - NestJS)

API Gateway Layer
• CORS • Rate Limiting • Request Validation

Authentication & Authorization
• JWT Strategy • Passport • Guards
• RolesGuard • PermissionsGuard

Business Modules

Users Module | Rentals Module | Credits Module
Cash Module | Roles Module | Audit Module
```

}

```
WebSocket Gateway (Socket.io)
• Real-time Notifications
• Room Management by Role
• Event Broadcasting

Services Layer
• Business Logic • Data Validation
• External Integrations • Email Service
```

```
┌──────────────────────────────────────────────────────────────┐
│                        CAPA DE DATOS                         │
│                   (PostgreSQL + Redis)                       │
└──────────────────────────────────────────────────────────────┘
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                  PostgreSQL Database                 │   │
│   │                                                      │   │
│   │   ┌─────────┐   ┌─────────┐   ┌─────────────┐        │   │
│   │   │ users   │   │ roles   │   │ permissions │        │   │
│   │   └─────────┘   └─────────┘   └─────────────┘        │   │
│   │                                                      │   │
│   │   ┌─────────┐   ┌─────────┐   ┌─────────┐          │   │
│   │   │ rentals │   │ credits │   │ cash_req │          │   │
│   │   └─────────┘   └─────────┘   └─────────┘          │   │
│   │                                                      │   │
│   │   ┌─────────┐   ┌─────────┐                         │   │
│   │   │ audit_log │ │ sessions │                        │   │
│   │   └─────────┘   └─────────┘                         │   │
│   │                                                      │   │
│   └──────────────────────────────────────────────────────┘   │
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │                     Redis Cache                      │   │
│   │   • Session Storage                                  │   │
│   │   • Rate Limiting Counters                           │   │
│   │   • Real-time Event Queue                            │   │
│   │   • Cached Queries (roles, permissions)              │   │
│   └──────────────────────────────────────────────────────┘   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

🌐 Arquitectura de Red (Servidor Local)

```
EMPRESA - RED LOCAL
```

└──────┘

📂 Estructura de Directorios del Proyecto

```
ferreteria-system/
│
├── frontend/                    # Aplicación React
│   ├── public/
│   │   ├── index.html
│   │   └── favicon.ico
│   │
│   ├── src/
│   │   ├── main.tsx             # Punto de entrada
│   │   ├── App.tsx              # Componente raíz
│   │   │
│   │   ├── routes/              # Configuración de rutas
│   │   │   ├── AppRouter.tsx
│   │   │   └── ProtectedRoute.tsx
│   │   │
│   │   ├── pages/               # Páginas principales
│   │   │   ├── Login.tsx
│   │   │   │
│   │   │   ├── admin/           # Dashboard Admin
│   │   │   │   ├── Dashboard.tsx
│   │   │   │   ├── UserManagement.tsx
│   │   │   │   ├── RoleManagement.tsx
│   │   │   │   └── AuditLogs.tsx
│   │   │   │
│   │   │   ├── secretaria/      # Dashboard Secretaria
│   │   │   │   ├── Dashboard.tsx
│   │   │   │   ├── RentalApprovals.tsx
│   │   │   │   ├── CreditApprovals.tsx
│   │   │   │   ├── CashApprovals.tsx
│   │   │   │   └── Reports.tsx
│   │   │   │
│   │   │   ├── colaborador/     # Dashboard Colaborador
│   │   │   │   ├── Dashboard.tsx
│   │   │   │   ├── RequestCredit.tsx
│   │   │   │   ├── RequestCash.tsx
│   │   │   │   └── MyRequests.tsx
│   │   │   │
```

```
│   │   │   └── encargado-maquinas/    # Dashboard Encargado
│   │   │       ├── Dashboard.tsx
│   │   │       ├── NewRental.tsx
│   │   │       ├── ActiveRentals.tsx
│   │   │       └── RentalHistory.tsx
│   │   │
│   │   ├── components/              # Componentes reutilizables
│   │   │   ├── layouts/
│   │   │   │   ├── DashboardLayout.tsx
│   │   │   │   ├── Sidebar.tsx
│   │   │   │   └── Topbar.tsx
│   │   │   │
│   │   │   ├── common/
│   │   │   │   ├── Button.tsx
│   │   │   │   ├── Input.tsx
│   │   │   │   ├── Modal.tsx
│   │   │   │   ├── Table.tsx
│   │   │   │   ├── Card.tsx
│   │   │   │   └── Loader.tsx
│   │   │   │
│   │   │   └── notifications/
│   │   │       ├── Toast.tsx
│   │   │       └── NotificationCenter.tsx
│   │   │
│   │   ├── hooks/                   # Custom hooks
│   │   │   ├── useAuth.ts
│   │   │   ├── useSocket.ts
│   │   │   ├── usePermissions.ts
│   │   │   └── useNotifications.ts
│   │   │
│   │   ├── services/                # Servicios API
│   │   │   ├── api.ts               # Configuración Axios
│   │   │   ├── auth.service.ts
│   │   │   ├── rentals.service.ts
│   │   │   ├── credits.service.ts
│   │   │   ├── cash.service.ts
│   │   │   ├── users.service.ts
│   │   │   └── socket.service.ts
│   │   │
│   │   ├── store/                   # Estado global (Zustand)
│   │   │   ├── authStore.ts
│   │   │   ├── notificationStore.ts
│   │   │   └── uiStore.ts
│   │   │
```

```
│   │   ├── types/                # TypeScript types
│   │   │   ├── user.types.ts
│   │   │   ├── rental.types.ts
│   │   │   ├── credit.types.ts
│   │   │   ├── cash.types.ts
│   │   │   └── api.types.ts
│   │   │
│   │   ├── utils/                # Utilidades
│   │   │   ├── formatters.ts
│   │   │   ├── validators.ts
│   │   │   └── constants.ts
│   │   │
│   │   └── styles/               # Estilos globales
│   │       └── globals.css
│   │
│   ├── .env.development
│   ├── .env.production
│   ├── vite.config.ts
│   ├── tsconfig.json
│   ├── tailwind.config.js
│   └── package.json
│
├── backend/                      # Aplicación NestJS
│   ├── src/
│   │   ├── main.ts               # Punto de entrada
│   │   │
│   │   ├── app.module.ts         # Módulo raíz
│   │   │
│   │   ├── modules/              # Módulos de negocio
│   │   │   │
│   │   │   ├── auth/             # Autenticación
│   │   │   │   ├── auth.module.ts
│   │   │   │   ├── auth.controller.ts
│   │   │   │   ├── auth.service.ts
│   │   │   │   ├── strategies/
│   │   │   │   │   ├── jwt.strategy.ts
│   │   │   │   │   └── local.strategy.ts
│   │   │   │   └── dto/
│   │   │   │       ├── login.dto.ts
│   │   │   │       └── register.dto.ts
│   │   │   │
│   │   │   ├── users/            # Gestión de usuarios
│   │   │   │   ├── users.module.ts
│   │   │   │   ├── users.controller.ts
```

```
│   │   │   │   ├── users.service.ts
│   │   │   │   └── dto/
│   │   │   │       ├── create-user.dto.ts
│   │   │   │       └── update-user.dto.ts
│   │   │   │
│   │   │   ├── roles/              # Gestión de roles
│   │   │   │   ├── roles.module.ts
│   │   │   │   ├── roles.controller.ts
│   │   │   │   ├── roles.service.ts
│   │   │   │   └── dto/
│   │   │   │
│   │   │   ├── rentals/            # Rentas de máquinas
│   │   │   │   ├── rentals.module.ts
│   │   │   │   ├── rentals.controller.ts
│   │   │   │   ├── rentals.service.ts
│   │   │   │   └── dto/
│   │   │   │       ├── create-rental.dto.ts
│   │   │   │       └── approve-rental.dto.ts
│   │   │   │
│   │   │   ├── credits/            # Créditos a clientes
│   │   │   │   ├── credits.module.ts
│   │   │   │   ├── credits.controller.ts
│   │   │   │   ├── credits.service.ts
│   │   │   │   └── dto/
│   │   │   │
│   │   │   ├── cash-requests/      # Solicitudes de efectivo
│   │   │   │   ├── cash-requests.module.ts
│   │   │   │   ├── cash-requests.controller.ts
│   │   │   │   ├── cash-requests.service.ts
│   │   │   │   └── dto/
│   │   │   │
│   │   │   ├── audit/              # Logs de auditoría
│   │   │   │   ├── audit.module.ts
│   │   │   │   ├── audit.service.ts
│   │   │   │   └── decorators/
│   │   │   │       └── audit-log.decorator.ts
│   │   │   │
│   │   │   └── notifications/      # Notificaciones tiempo real
│   │   │       ├── notifications.module.ts
│   │   │       ├── notifications.gateway.ts
│   │   │       └── notifications.service.ts
│   │   │
│   │   ├── common/                 # Código compartido
│   │   │   ├── guards/
```

```
│   │   │   │   ├── jwt-auth.guard.ts
│   │   │   │   ├── roles.guard.ts
│   │   │   │   └── permissions.guard.ts
│   │   │   │
│   │   │   ├── decorators/
│   │   │   │   ├── roles.decorator.ts
│   │   │   │   ├── permissions.decorator.ts
│   │   │   │   └── current-user.decorator.ts
│   │   │   │
│   │   │   ├── filters/
│   │   │   │   └── http-exception.filter.ts
│   │   │   │
│   │   │   ├── interceptors/
│   │   │   │   ├── logging.interceptor.ts
│   │   │   │   └── transform.interceptor.ts
│   │   │   │
│   │   │   └── pipes/
│   │   │       └── validation.pipe.ts
│   │   │
│   │   ├── prisma/                 # Prisma ORM
│   │   │   ├── prisma.module.ts
│   │   │   ├── prisma.service.ts
│   │   │   └── schema.prisma         # Schema de BD
│   │   │
│   │   ├── config/                 # Configuración
│   │   │   ├── app.config.ts
│   │   │   ├── database.config.ts
│   │   │   └── jwt.config.ts
│   │   │
│   │   └── logger/                 # Winston logger
│   │       └── winston.config.ts
│   │
│   ├── prisma/
│   │   ├── migrations/               # Migraciones de BD
│   │   └── seed.ts                 # Datos iniciales
│   │
│   ├── test/                       # Tests
│   │   ├── unit/
│   │   └── e2e/
│   │
│   ├── .env
│   ├── .env.example
│   ├── nest-cli.json
│   ├── tsconfig.json
```

```
│   └── package.json
│
├── docs/                        # Documentación
│   ├── API.md
│   ├── DEPLOYMENT.md
│   ├── DATABASE.md
│   └── ARCHITECTURE.md
│
├── scripts/                     # Scripts de utilidad
│   ├── backup.sh
│   ├── deploy.sh
│   └── seed-database.sh
│
├── .gitignore
├── README.md
└── docker-compose.yml               # Opcional para desarrollo
```

```
┌──────────────────────────────────────────────────────────────┐
│ 1. AUTENTICACIÓN                                             │
│ • JWT tokens (access + refresh)                              │
│ • Contraseñas hasheadas (bcrypt, 12 rounds)                  │
│ • Tokens expiración: 15min (access), 7 días (refresh)        │
└──────────────────────────────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────────────────────┐
│ 2. AUTORIZACIÓN                                              │
│ • Guards: JwtAuthGuard → verificar token                     │
│ • Guards: RolesGuard → verificar rol                         │
│ • Guards: PermissionsGuard → permisos granulares             │
└──────────────────────────────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────────────────────┐
│ 3. VALIDACIÓN                                                │
│ • DTOs con class-validator                                   │
│ • Sanitización de inputs                                     │
│ • Validación tipos (TypeScript)                              │
└──────────────────────────────────────────────────────────────┘
        │
        ▼
```

```
┌──────────────────────────────────────────────────────────────┐
│ 4. RATE LIMITING                                             │
│ • Límite por rol (admin: 100/min, user: 30/min)              │
│ • Prevención DDoS                                            │
│ • Throttling con Redis                                       │
└──────────────────────────────────────────────────────────────┘
          │
          ▼
┌──────────────────────────────────────────────────────────────┐
│ 5. AUDITORÍA                                                 │
│ • Logs de todas las acciones críticas                        │
│ • Registro de IP, timestamp, usuario                         │
│ • Trazabilidad completa                                      │
└──────────────────────────────────────────────────────────────┘
          │
          ▼
┌──────────────────────────────────────────────────────────────┐
│ 6. ENCRIPTACIÓN                                              │
│ • HTTPS/TLS (SSL certificado)                                │
│ • Datos sensibles encriptados en BD                          │
│ • Variables de entorno (.env)                                │
└──────────────────────────────────────────────────────────────┘
```

Arquitectura Detallada (la expandida):

```
Frontend (React + Vite)
    ├── Pages (Admin, Secretaria, Colaborador, Encargado)
    ├── Components (Layouts, Common, Notifications)
    ├── Services (API calls)
    └── Hooks (useAuth, useSocket)
      ↕ HTTP REST + WebSocket
Backend (NestJS)
    ├── Auth Module
    ├── Users Module
    ├── Rentals Module
    ├── Credits Module
    ├── Cash Requests Module
    ├── WebSocket Gateway
    └── Guards & Middleware
      ↕ Prisma ORM
Database (PostgreSQL + Redis)
    ├── Tables (users, roles, rentals, credits, etc)
```

```
└── Redis (sessions, cache)
```

La Arquitectura Base (Tu Diagrama) - CONFIRMADA:

```
┌─────────────────────────────────────────────────────────┐
│   FRONTEND (React + Vite + TS)        │ ← SPA moderna
│                                       │
│ • SPA (Single Page Application)       │ ← Una sola página
│ • React Router para navegación        │ ← Rutas cliente
│ • Auth Context para sesión            │ ← Estado global auth
│ • Socket.io para notificaciones real  │ ← Tiempo real
│                                       │
└──────────────────┬──────────────────────────────────────┘
                   │
                   │ HTTP REST + WebSocket      ← Comunicación
                   │
┌──────────────────▼──────────────────────────────────────┐
│   BACKEND (NestJS + TypeScript)       │ ← API profesional
│                                       │
│ • API REST endpoints                  │ ← CRUD operations
│ • WebSocket Gateway (Socket.io)       │ ← Notificaciones
│ • JWT Authentication                  │ ← Seguridad
│ • RBAC (Roles + Permissions)          │ ← Control acceso
│ • Audit Logs                          │ ← Trazabilidad
│                                       │
└──────────────────┬──────────────────────────────────────┘
                   │
                   │ Prisma ORM                 ← ORM moderno
                   │
┌──────────────────▼──────────────────────────────────────┐
│   PostgreSQL + Redis                  │ ← Bases de datos
│                                       │
│ PostgreSQL:                           │ ← BD principal
│ • Users, Roles, Permissions           │
│ • Rentals, Credits, Cash              │
│ • Audit Logs                          │
│                                       │
│ Redis:                                │ ← Caché/Sesiones
│ • User sessions (JWT blacklist)       │
│ • Real-time notifications queue       │
│ • Cache de consultas frecuentes       │
```

```
│                          │
└──────────────────────────────────────────────┘
```

📋 Resumen: ¿Qué sigue igual?

Componente	Status	Tecnología
Frontend Framework	✅ Igual	React 18 + Vite
Frontend Language	✅ Igual	TypeScript
Frontend Router	✅ Igual	React Router DOM
Frontend State	✅ Igual	Context API / Zustand
Real-time Client	✅ Igual	Socket.io-client
Backend Framework	✅ Igual	NestJS 10
Backend Language	✅ Igual	TypeScript
Authentication	✅ Igual	JWT + Passport
Authorization	✅ Igual	RBAC (Roles + Permissions)
Real-time Server	✅ Igual	Socket.io
ORM	✅ Igual	Prisma 5
Primary Database	✅ Igual	PostgreSQL 15+
Cache/Sessions	✅ Igual	Redis 7
Logging	✅ Igual	Winston
Deployment	✅ Igual	Servidor propio Ubuntu

🎯 La diferencia entre los diagramas:

Tu diagrama (Simplificado):

- **Propósito:** Vista de alto nivel para entender la arquitectura general
- **Audiencia:** Cliente, stakeholders, overview rápido

- **Nivel:** Conceptual

Mi diagrama expandido:

- **Propósito:** Vista detallada para implementación
- **Audiencia:** Desarrolladores
- **Nivel:** Técnico/Implementación

Ambos representan EL MISMO SISTEMA, solo que con diferente nivel de detalle.

App.tsx (React Router - Controla qué mostrar):

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Ruta pública */}
        <Route path="/login" element={<Login />} />

        {/* Rutas de Admin */}
        <Route path="/admin/dashboard" element={<AdminDashboard />} />
        <Route path="/admin/users" element={<UserManagement />} />

        {/* Rutas de Secretaria */}
        <Route path="/secretaria/dashboard" element={<SecretariaDashboard />} />
        <Route path="/secretaria/rentas" element={<RentalApprovals />} />

        {/* Rutas de Colaborador */}
        <Route path="/colaborador/dashboard" element={<ColaboradorDashboard />} />
        <Route path="/colaborador/creditos" element={<RequestCredit />} />

        {/* Rutas de Encargado */}
        <Route path="/encargado/dashboard" element={<EncargadoDashboard />} />
        <Route path="/encargado/rentas" element={<NewRental />} />
      </Routes>
    </BrowserRouter>
  );
}
```

React Router decide QUÉ componente mostrar según la URL.

🖼️ Visualización:

Lo que ve el usuario (4 páginas diferentes):

```
┌──────────────────────────────────────────┐
│ LOGIN                                    │
│ [Email]   [Password]   [Entrar] │
└──────────────────────────────────────────┘

Usuario ingresa como ADMIN
   ↓ Cambio instantáneo (sin recarga)

┌──────────────────────────────────────────┐
│ DASHBOARD ADMINISTRADOR                  │
│ ┌──────────┐ ┌──────────┐ ┌──────────┐   │
│ │Users │ │Roles │ │Logs │ │          │
│ └──────────┘ └──────────┘ └──────────┘   │
└──────────────────────────────────────────┘

Usuario ingresa como SECRETARIA
   ↓ Cambio instantáneo (sin recarga)

┌──────────────────────────────────────────┐
│ DASHBOARD SECRETARIA                     │
│ ┌──────────┐ ┌──────────┐ ┌──────────┐   │
│ │Rentas │ │Créditos│ │Efectivo│ │
│ └──────────┘ └──────────┘ └──────────┘   │
└──────────────────────────────────────────┘

Usuario ingresa como COLABORADOR
   ↓ Cambio instantáneo (sin recarga)

┌──────────────────────────────────────────┐
│ DASHBOARD COLABORADOR                    │
│ ┌──────────────┐ ┌──────────────┐        │
│ │Solicitar │ │Solicitar │       │
│ │Crédito   │ │Efectivo  │       │
│ └──────────────┘ └──────────────┘        │
└──────────────────────────────────────────┘
```

Lo que realmente pasa técnicamente:

```
┌─────────────────────────────────────────────────┐
│ index.html                                      │
│ ┌──────────────────────────────────────────┐    │
│ │ <div id="root">                          │    │
│ │                                          │    │
│ │   [Aquí React cambia                     │    │
│ │    el contenido según                    │    │
│ │    la ruta]                              │    │
│ │                                          │    │
│ │   Login Component                        │    │
│ │     ↓ cambia a                           │    │
│ │   AdminDashboard Component               │    │
│ │     ↓ cambia a                           │    │
│ │   SecretariaDashboard                    │    │
│ │     ↓ cambia a                           │    │
│ │   ColaboradorDashboard                   │    │
│ │                                          │    │
│ └──────────────────────────────────────────┘    │
└─────────────────────────────────────────────────┘
```

Extensiones recomendadas para VS Code:

-

ESLint
Prettier
Prisma
TypeScript and JavaScript Language Features
Tailwind CSS IntelliSense
Thunder Client (alternativa a Postman)